The Hidden Cost of Volatility

Research Questions:

1. How much wealth is destroyed by volatility and leverage, even despite a positive expectancy?

2. How much does volatility reduce long-run wealth?

3. What happens when leverage is introduced?

4. At what point does leverage destroy growth, despite increasing expected returns?

Importing Necessary Libraries

In [40]:

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Setup

We work with a portfolio of assets that earns a random return, r_t. Wealth updates as $V_t = V_{t-1}(1 + r_t)$. This equation, is everything. Compounding, geometric returns, volatility drag, and leverage comes from this one line.

Setup:

1. Initial wealth = 100
2. Mean annual return = 10%
3. Volatility = 20%
4. Horizon = 20 years

In [41]:

```
V0 = 100        # Starting wealth
mu = 0.10       # Positive expectancy - If I simulate many years, the average yearly return should be around 20%
sigma = 0.20    # Volatility - Describes how much returns fluctuate around the mean
N = 20          # Time in years
```

In [42]:

```
returns = np.random.normal(loc = mu, scale = sigma, size = N)
```

Every year, we draw a random return from $r_t \sim N(\mu, \sigma^2)$.

In [43]:

```
wealth = [V0]
```

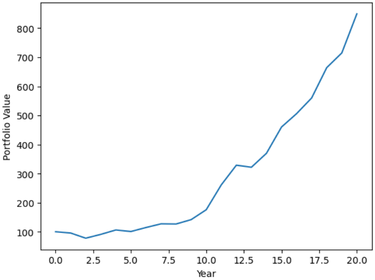
In [44]:

```
for i in range(0, N):

    new_wealth = wealth[i] * (1 + returns[i])

    wealth.append(new_wealth)

print(wealth)
```

```
[100, 95.62043533251551, 77.79281540654914, 90.94852619417108, 106.09495455140596, 100.79248339435267, 114.5070175804689, 127.20158141555177, 126.
63158111305695, 141.66304992094524, 175.63796456356656, 261.2609937269936, 328.8852225505253, 321.81566277956813, 369.87833924517514, 460.26960391
329584, 506.24318882130126, 560.2030519141069, 664.454555932068, 714.8888027587851, 849.554956691726]
```

In [45]:

```
plt.plot(wealth)
plt.xlabel("Year")
plt.ylabel("Portfolio Value")
plt.show()
```



2. Simulations

In [46]:

```
n_sims = 10_000     # We are creating 10,000 different possible universes
```

We are creating 10,000 different possible universes. Each universe has different returns, different welath paths, but the same mean, and same volatility.

In [47]:

```
returns = np.random.normal(loc = mu, scale = sigma, size = (n_sims, N))

print(returns.shape)
```

```
(10000, 20)
```

In [48]:

```python
wealth = np.zeros((n_sims, N + 1))

wealth[ : , 0] = V0

print(wealth)
```

```
[[100.   0.   0. ...   0.   0.   0.]
 [100.   0.   0. ...   0.   0.   0.]
 [100.   0.   0. ...   0.   0.   0.]
 ...
 [100.   0.   0. ...   0.   0.   0.]
 [100.   0.   0. ...   0.   0.   0.]
 [100.   0.   0. ...   0.   0.   0.]]
```

In [49]:

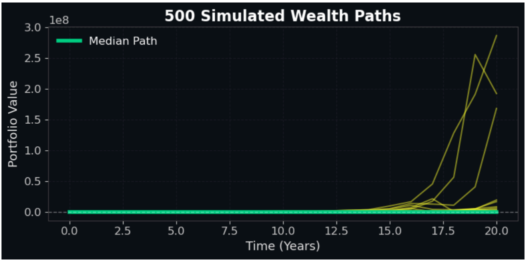
```python
for t in range(1, N + 1):
    wealth[ : , t] = wealth[ :, t - 1] * (1 + returns[ :, t - 1])

print(wealth)
```

```
[[ 100.           98.21757889   98.09111759 ...  535.55445284
   658.6592747    857.67349101]
 [ 100.          115.07924334  132.51830447 ...  991.52684559
  1189.4782408   1196.31432081]
 [ 100.           62.26756705   68.11339284 ...  182.48397281
   207.02174138  243.72486835]
 ...
 [ 100.          108.76059376  134.04224476 ...  219.26954133
   298.62696547  502.22921773]
 [ 100.           70.78495565   82.45573955 ...  194.41395825
   201.90160255  263.06908342]
 [ 100.          106.57239558   96.51602699 ...  115.51610373
   137.88034675  166.67913094]]
```

3. Visualizations

In [98]:

```python
plt.style.use("dark_background")

plt.rcParams.update({
    "figure.figsize": (8, 4),
    "axes.facecolor": "#0e1117",
    "figure.facecolor": "#0e1117",
    "axes.edgecolor": "#444",
    "axes.labelcolor": "#e6e6e6",
    "xtick.color": "#cccccc",
    "ytick.color": "#cccccc",
    "grid.color": "#2a2f3a",
    "grid.linestyle": "--",
    "grid.alpha": 0.4,
    "font.size": 12
})

n_paths = 300

for i in range(n_paths):
    plt.plot(
        wealth[i],
        color="#FFFF33",
        alpha=0.5,
        linewidth=1.5
    )


median_path = np.median(wealth, axis=0)

plt.plot(
    median_path,
    color="#00d084",
    linewidth=4,
    label="Median Path"
)


plt.axhline(
    y=100,
    linestyle="--",
    color="white",
    alpha=0.4,
    linewidth=1
)

plt.title(
    "500 Simulated Wealth Paths",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Time (Years)", fontsize=13)
plt.ylabel("Portfolio Value", fontsize=13)

plt.legend(frameon=False)
plt.grid(True)
plt.tight_layout()

plt.show()
```



In [92]:
```
arith_return = returns.mean()

print(arith_return)
```
```
0.6487857897435741
```

In [52]:
```
final_wealth = wealth[ : , -1]
```

In [53]:
```
geo_return = (final_wealth / V0) ** (1 / N) - 1

print(geo_return.mean())
```
```
0.0818245694108922
```

In [54]:
```
print("Average Final Wealth:", final_wealth.mean().round(2))

print("Average Median Wealth:", np.median(final_wealth).round(2))
```
```
Average Final Wealth: 668.31
Average Median Wealth: 483.52
```

Average wealth is higher than median wealth, this is because the average tends to get pulled up by a few extreme price paths. The average can look fantastic while the typical investor barely grows wealth.

Simulation Function

In [55]:
```
def simulate_wealth(V0, mu, sigma, N, n_sims, leverage=1):

    # Generate returns
    returns = np.random.normal(loc=mu, scale=sigma, size=(n_sims, N))

    # Apply leverage
    returns = leverage * returns

    # Prevent returns below -100%
    returns = np.maximum(returns, -0.99)

    # Wealth matrix
    wealth = np.zeros((n_sims, N+1))
    wealth[:,0] = V0

    # Compound wealth
    for t in range(1, N+1):
        wealth[:,t] = (wealth[:,t-1] * (1 + returns[:,t-1]))

    return returns, wealth
```

In [56]:
```
returns, wealth = simulate_wealth(V0=100, mu=0.10, sigma=0.20, N=20, n_sims=10000, leverage=1)
```

How much wealth is destroyed by volatility?

Wealth destroyed = wealth (sigma = 0) - wealth (sigma > 0)

In [57]:
```
_, wealth_no_vol = simulate_wealth(V0=100, mu=0.10, sigma=0, N=20, n_sims=10000)
```

In [58]:
```
_, wealth_vol = simulate_wealth(V0=100, mu=0.10, sigma=0.20, N=20, n_sims=10000)
```

In [59]:
```
median_no_vol = np.median(wealth_no_vol[:,-1])

median_vol = np.median(wealth_vol[:,-1])

print("Median wealth (No volatility):", median_no_vol.round(2))

print("Median wealth (with volatility):", median_vol.round(2))

print("Wealth destroyed by volatility:", (median_no_vol - median_vol).round(2))
```
```
Median wealth (No volatility): 672.75
Median wealth (with volatility): 484.63
Wealth destroyed by volatility: 188.12
```

It is true that volatility makes returns more uncertain, but the deeper insight is that volatility permanently reduces the rate at which wealth compounds. This is the true cost of volatility.

Volatility does not merely make outcomes uncertain, it permanently reduces the rate at which wealth grows & compounds.

How much does volatility reduce long-run wealth?

In [60]:

```python
sigmas = [0, 0.1, 0.2, 0.3, 0.4, 0.5]
```

In [61]:

```python
median_wealths = []

geo_returns = []
```

In [62]:

```python
for sigma in sigmas:

    returns, wealth = simulate_wealth(V0=100, mu=0.10, sigma=sigma, N=20, n_sims=10000)

    final_wealth = wealth[:,-1]

    median_wealths.append(np.median(final_wealth))

    geo = ((final_wealth/100) ** (1/20) - 1)

    geo_returns.append(geo.mean())
```

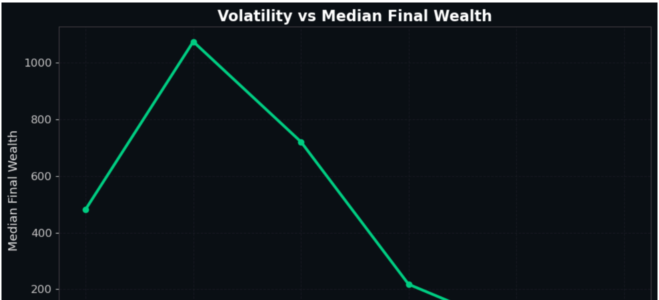
In [63]:

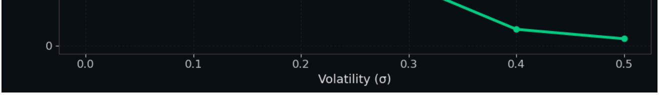
```python
print("Volatilities")

print(sigmas)

print()

print("Median Wealth")

print(median_wealths)

print()

print("Mean Geometric Return")

print(geo_returns)
```

```
Volatilities
[0, 0.1, 0.2, 0.3, 0.4, 0.5]

Median Wealth
[672.7499949325611, 618.6144650445385, 476.68295230082174, 296.9773074756258, 130.55990841733092, 31.13211765408556]

Mean Geometric Return
[0.10000000000000009, 0.09556021568756043, 0.08159029636770757, 0.05529042100512888, 0.006389540189376487, -0.06409740822784248]
```

In [97]:

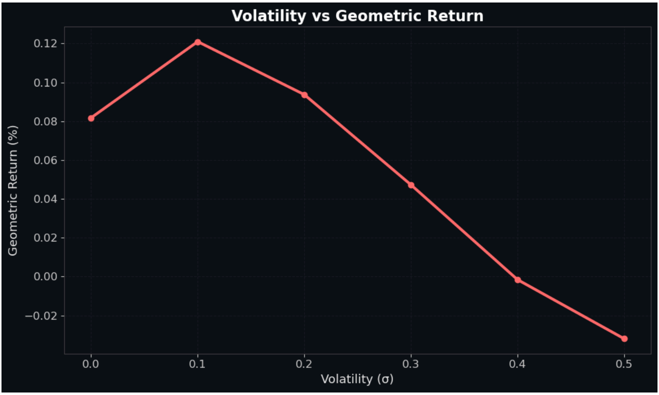
```python
plt.style.use("dark_background")

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "axes.facecolor": "#0e1117",
    "figure.facecolor": "#0e1117",
    "axes.edgecolor": "#444",
    "axes.labelcolor": "#e6e6e6",
    "xtick.color": "#cccccc",
    "ytick.color": "#cccccc",
    "grid.color": "#2a2f3a",
    "grid.linestyle": "--",
    "grid.alpha": 0.4,
    "font.size": 12
})

plt.plot(
    sigmas,
    median_wealths,
    marker="o",
    linewidth=3,
    color="#00d084"
)

plt.title(
    "Volatility vs Median Final Wealth",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Volatility (σ)", fontsize=13)
plt.ylabel("Median Final Wealth", fontsize=13)

plt.grid(True)
plt.tight_layout()

plt.show()
```





In [78]:

```
plt.style.use("dark_background")

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "axes.facecolor": "#0e1117",
    "figure.facecolor": "#0e1117",
    "axes.edgecolor": "#444",
    "axes.labelcolor": "#e6e6e6",
    "xtick.color": "#cccccc",
    "ytick.color": "#cccccc",
    "grid.color": "#2a2f3a",
    "grid.linestyle": "--",
    "grid.alpha": 0.4,
    "font.size": 12
})

# Plot
plt.plot(
    sigmas,
    geo_returns,
    marker="o",
    linewidth=3,
    color="#ff6b6b"
)

plt.title(
    "Volatility vs Geometric Return",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Volatility (σ)", fontsize=13)
plt.ylabel("Geometric Return (%)", fontsize=13)

plt.grid(True)
plt.tight_layout()

plt.show()
```



Volatility attacks the growth rate first, the damage to wealth comes later as a consequence of that reduced growth rate. The positive expectancy (mu) never changed, the average yearly return was held fixed.

Most people think, "If my strategy earns 10% on average, I should become wealthy." But this simulation says, "Not necessarily!" If I see a strategy with a positive expectancy of 15%, I should immediately ask - "What is the volatility?" Because, 15% alone tells me nothing

In a multiplicative world, maximizing rexpected returns, is not the same as maximizing wealth. Maximizing your sharpe ratio and maximizing your wealth are not the same optimization problem! A strategy can have a good sharpe, positive expectancy, and an attractive backtest, but it can have a negative geometric growth.

Sharpe tells you how attractive the edge is. Geometric growth tells you how much your wealth grows.

Can wealth reduce even if positive expectancy reduces due to leverage? At what point does leverage destroy wealth, despite increasing expected returns?

In [66]:

```
mu = 0.10

sigma = 0.20

V0 = 100
```

```
N = 20

n_sims = 10000
```

In [67]:

```
leverages = [1, 2, 3, 4, 5, 6] # Only leverage changes
```

In [68]:

```
arith_returns = []

geo_returns = []

median_wealths = []

prob_ruin = []
```

In [69]:

```
for L in leverages:

    returns, wealth = simulate_wealth(V0=100, mu=0.10, sigma=0.20, N=20, n_sims=10000, leverage=L)

    # Arithmetic return
    arith_returns.append(returns.mean())
    final_wealth = wealth[:, -1]

    # Geometric return
    geo = ((final_wealth/100) ** (1/20) - 1)
    geo_returns.append(geo.mean())

    # Median wealth
    median_wealths.append(np.median(final_wealth))

    # Probability of ruin
    ruin = (final_wealth < 10)

    prob_ruin.append(ruin.mean())
```

In [70]:

```
print("Leverages")

print(leverages)

print()

print("Arithmetic Returns")

print(arith_returns)

print()

print("Geometric Returns")

print(geo_returns)

print()

print("Median Wealth")

print(median_wealths)

print()

print("Probability of Ruin")

print(prob_ruin)
```

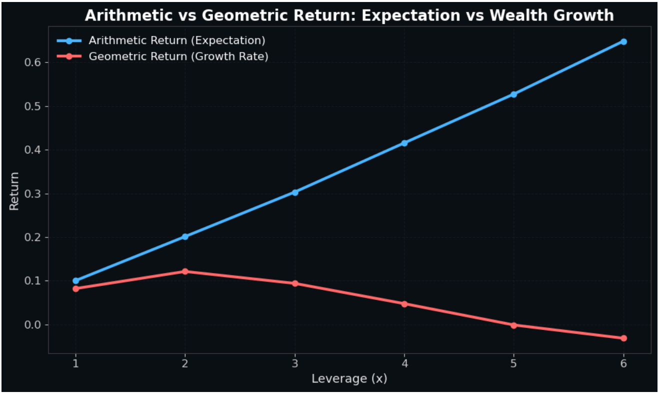
```
Leverages
[1, 2, 3, 4, 5, 6]

Arithmetic Returns
[0.09989889786251223, 0.20089662623991295, 0.3028293447694747, 0.41546367253778044, 0.5274718340218842, 0.6487857897435741]

Geometric Returns
[0.08175344807461955, 0.1209808935022751, 0.093696728536305, 0.0472501926563222, -0.0016070925408047509, -0.03202310203723939]

Median Wealth
[481.7160795761569, 1073.2977265285426, 720.1438660078652, 217.62363519269962, 59.042504846128594, 25.661723271563808]

Probability of Ruin
[0.0, 0.0275, 0.1557, 0.2934, 0.393, 0.4488]
```

Arithmetic return keeps increasing as you keep leveraging, but more leverage does not always mean better growth.

In [77]:

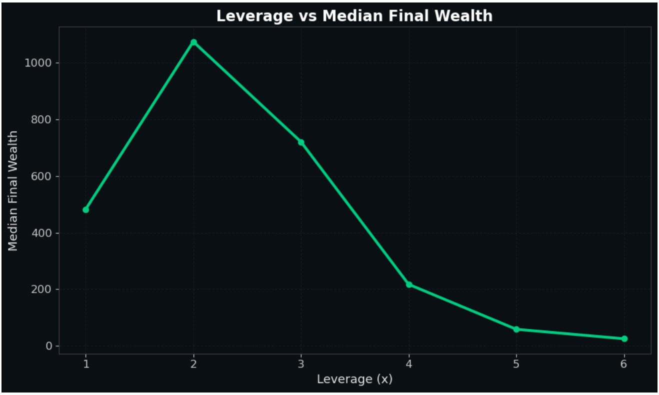
```
plt.style.use("dark_background")

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "axes.facecolor": "#0e1117",
    "figure.facecolor": "#0e1117",
    "axes.edgecolor": "#444",
    "axes.labelcolor": "#e6e6e6",
    "xtick.color": "#cccccc",
    "ytick.color": "#cccccc",
    "grid.color": "#2a2f3a",
    "grid.linestyle": "--",
    "grid.alpha": 0.4,
    "font.size": 12
})

plt.plot(
    leverages,
    arith_returns,
    marker="o",
    linewidth=3,
    color="#4db8ff",
    label="Arithmetic Return (Expectation)"
)

plt.plot(
    leverages,
```

```
    geo_returns,
    marker="o",
    linewidth=3,
    color="#ff6b6b",
    label="Geometric Return (Growth Rate)"
)

plt.title("Arithmetic vs Geometric Return: Expectation vs Wealth Growth",
          fontsize=16, fontweight="bold")

plt.xlabel("Leverage (x)", fontsize=13)
plt.ylabel("Return", fontsize=13)

plt.grid(True)

plt.legend(frameon=False)

plt.tight_layout()

plt.show()
```



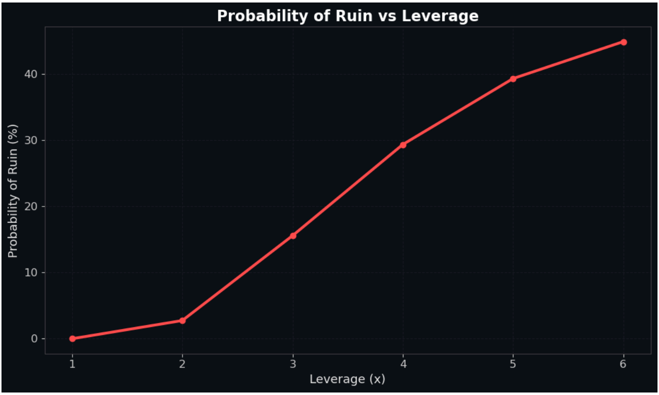
In [76]:

```
plt.style.use("dark_background")

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "axes.facecolor": "#0e1117",
    "figure.facecolor": "#0e1117",
    "axes.edgecolor": "#444",
    "axes.labelcolor": "#e6e6e6",
    "xtick.color": "#cccccc",
    "ytick.color": "#cccccc",
    "grid.color": "#2a2f3a",
    "grid.linestyle": "--",
    "grid.alpha": 0.4,
    "font.size": 12
})

plt.plot(
    leverages,
    median_wealths,
    marker="o",
    linewidth=3,
    color="#00d084"
)

plt.title("Leverage vs Median Final Wealth", fontsize=16, fontweight="bold")
plt.xlabel("Leverage (x)", fontsize=13)
plt.ylabel("Median Final Wealth", fontsize=13)

plt.grid(True)
plt.tight_layout()

plt.show()
```



In [75]:

```
plt.style.use("dark_background")

plt.rcParams.update({
    "figure.figsize": (10, 6),
    "axes.facecolor": "#0e1117",
    "figure.facecolor": "#0e1117",
    "axes.edgecolor": "#444",
    "axes.labelcolor": "#e6e6e6",
    "xtick.color": "#cccccc",
    "ytick.color": "#cccccc",
    "grid.color": "#2a2f3a",
    "grid.linestyle": "--",
    "grid.alpha": 0.4,
    "font.size": 12
})

prob_ruin_pct = [x * 100 for x in prob_ruin]

plt.plot(
    leverages,
    prob_ruin_pct,
    marker="o",
    linewidth=3,
    color="#ff4d4d"
)

plt.title("Probability of Ruin vs Leverage", fontsize=16, fontweight="bold")
plt.xlabel("Leverage (x)", fontsize=13)
plt.ylabel("Probability of Ruin (%)", fontsize=13)

plt.grid(True)

plt.tight_layout()

plt.show()
```



Summary

This project aimed to answer three main question addressed on the 'True Cost of Volatility':

1. How much wealth is destroyed by volatility?

- The average final wealth (with volatility) can look fantastic while the typical investor still ends up losing wealth. This is because the average paints a misleading picture as it is pulled up by a few extreme price paths.

1. How much does volatility reduce long-run wealth?

- Not only does volatility make returns more uncertain, but it also permanently reduces the rate at which wealth grows. This is the true cost of volatility. Volatility does not merely make outcomes uncertain, but it permanently destroys your wealth, even if you have a positive expectancy.
- Volatility attacks growth rate first, wealth reduces due to the consequence of reduced growth rate.
- Maximizing expected returns is not the same optimization problem as maximizing growth rate of your wealth.
- A strategy with even a 15% positive expectancy tells me nothing, unless I know the volatility!

1. At what point does leverage destroy wealth despite increasing expected returns

- Expected returns will increase as you take on more leverage, but that will lead to reduced wealth after a point. Wealth can reduce, even if expected returns increase due to leverage. More leverage does not mean increased growth.